

COMPLETE PIPELINE — FROM REDDIT TO MARKET-GAP JSON

I'll structure this like a real production system design.

PHASE 0 — GOAL

Your system should answer:

“What repeated technology/product frustrations are users discussing that indicate a potential business opportunity?”

NOT:

- general sentiment analysis
- generic Reddit scraping
- random AI summaries

You are building:

a Product Pain Intelligence Pipeline.

PHASE 1 — DATA INGESTION

STEP 1 — Connect to Reddit

Use:

- [Reddit API](#)
 - [PRAW](#)
-

What Happens Here

Your system:

- authenticates with Reddit
 - fetches posts/comments
 - processes them one-by-one
-

Example

```
reddit.subreddit("OpenAI").hot(limit=100)
```

You are NOT downloading all Reddit.

Only:

- targeted subreddits
 - limited recent posts
-

STEP 2 — Target Subreddit Selection

You manually define:

```
TARGET_SUBREDDITS = [  
    "OpenAI",  
    "ClaudeAI",  
    "LocalLLaMA",  
    "MachineLearning",  
    "SaaS",  
    "startups",  
    "webdev",  
    "programming"  
]
```

Purpose:

- reduce noise immediately
 - stay focused on tech/product discussions
-

PHASE 2 — STREAM FILTERING PIPELINE

VERY IMPORTANT:

You DO NOT store posts yet.

Each post goes through filters first.

STEP 3 — Initial Metadata Filtering

Cheap filtering first.

Checks

A. Minimum Text Length

Discard:

"lol"

"same"

"this sucks"

Example:

```
if len(post_text) < 50:  
    discard()
```

B. Upvote Threshold

Ignore low-engagement noise.

Example:

```
if upvotes < 5:  
    discard()
```

C. Recency Filter

Only recent posts:

last 30 days

Why?

You want:

- current market pain
 - current product issues
-

STEP 4 — TECH RELEVANCE FILTER

Now determine:

“Is this actually a tech/product discussion?”

IMPORTANT

This is NOT sentiment analysis yet.

This is:

semantic domain classification.

APPROACH

Layer 1 — Cheap Keyword Filter

Example:

```
TECH_KEYWORDS = [  
    "AI",  
    "API",  
    "tool",  
    "software",  
    "model",  
    "app",  
    "platform",  
    "LLM",  
    "plugin",  
    "SaaS"  
]
```

If no matches:

→ discard early.

STEP 5 — Lightweight NLP Classifier

Now run a small model.

NOT GPT.

Use:

- DistilBERT
 - MiniLM
 - zero-shot classifier
-

Input

"Cursor completely freezes on large repositories."

Output

```
{  
  "label": "tech_product_discussion",  
  "confidence": 0.96  
}
```

Keep only:

confidence > 0.8

PHASE 3 — PRODUCT + BUSINESS DETECTION

STEP 6 — Product/Company Extraction

Now identify:

- product names
 - company names
-

Example

Input:

"Gemini hallucinates during coding tasks."

Output:

```
{  
  "product": "Gemini",
```

```
"company": "Google"
}
```

HOW?

Use:

- spaCy NER
 - custom dictionary
 - LLM extraction later
-

MVP APPROACH

Maintain:

```
KNOWN_PRODUCTS = {
    "Gemini": "Google",
    "Claude": "Anthropic",
    "Cursor": "Cursor",
    "ChatGPT": "OpenAI"
}
```

STEP 7 — Business Validation

Now verify:

“Is this commercially meaningful?”

WHY?

You don't care about:

- tiny abandoned apps
- hobby projects

You want:

- established products

- market-relevant companies
-

MVP LOGIC

Check:

- company exists
 - known company
 - enough market presence
-

LATER IMPROVEMENTS

Integrate:

- [Crunchbase](#)
- [Clearbit](#)

to get:

- funding
 - employee count
 - company size
-

PHASE 4 — FRUSTRATION DETECTION

STEP 8 — Negative Sentiment Filter

Now detect:

- frustration
 - complaints
 - workflow pain
-

Example

GOOD SIGNAL:

"We had to manually verify every Gemini output."

BAD SIGNAL:

"I like Gemini."

WHAT TO USE

MVP:

- VADER
- TextBlob

Better:

- transformer sentiment models

Output

```
{  
  "sentiment": -0.81,  
  "frustration_detected": true  
}
```

STEP 9 — WORKFLOW/BUSINESS PAIN DETECTION

VERY IMPORTANT.

This is stronger than simple negativity.

Example

Weak:

"This UI sucks."

Strong:

"Our team cannot deploy this in production because reliability is poor."

This indicates:

business pain.

HOW?

Use:

- keyword patterns
- later LLM extraction

Look for:

```
[  
  "manually",  
  "workaround",  
  "our company",  
  "production",  
  "enterprise",  
  "compliance",  
  "workflow"  
]
```

PHASE 5 — STORE FILTERED RAW JSON

NOW store.

Only after all filters pass.

STEP 10 — Create Raw Intelligence JSON

Example:

```
{  
  "post_id": "abc123",  
  "subreddit": "OpenAI",  
  "title": "Gemini hallucinates badly",  
  "body": "Gemini invents APIs during coding.",  
  "upvotes": 142,  
  "timestamp": "2026-05-16",  
  "product": "Gemini",  
}
```

```
"company": "Google",  
"tech_confidence": 0.95,  
"sentiment_score": -0.82,  
"business_relevance": 0.89,  
"workflow_pain_detected": true  
}
```

THIS becomes your:

validated intelligence layer